

An inefficient approach is:

```
users = User.objects.all()
```

And an optimised approach can be:

```
users = User.objects.only('id', 'username', 'email')
```

...or:

```
users = User.objects.values('id', 'username', 'email')
```

The 'only' method returns model instances but defers other fields. The 'values' method returns dictionaries, which works excellently for API serialisation where you're building JSON responses.

Eliminating N+1 queries with `select_related` handles foreign key relationships efficiently.

N+1 queries are bad:

```
orders = Order.objects.all()
for order in orders:
    print(order.customer.name) # Separate query for EACH order
```

...while a single JOIN query is good:

```
orders = Order.objects.select_related('customer')
for order in orders:
    print(order.customer.name) # No additional queries
```

Use `select_related` for foreign key and one-to-one relationships. For many-to-many and reverse foreign key relationships, use `prefetch_related`.

Here's the code for many-to-many relationships:

```
books = Book.objects.prefetch_related('authors')
```

For complex nested relationships use:

```
orders = Order.objects.select_related(
    'customer', 'shipping_address'
).prefetch_related('items__product__category')
```

Leveraging database indexes strategically

Indexes are data structures that databases use to find rows quickly without scanning entire tables. Without indexes, PostgreSQL must examine every row to find matches—acceptable for small tables with hundreds of rows, disastrous for millions of records.

Adding indexes in Django models is straightforward:

```
class Product(models.Model):
```

```
    name = models.CharField(max_length=200)
    sku = models.CharField(max_length=50, db_index=True)
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
    class Meta:
        indexes = [ models.Index(fields=['category', 'price']),
                    models.Index(fields=['-created_at', 'category']), ]
```

The `db_index` parameter creates a single-column index. The `Meta.indexes` list creates multi-column indexes for queries filtering on multiple fields.

Index considerations require balancing performance gains against costs. Indexes consume disk space and slow down INSERT, UPDATE, and DELETE operations.

Indexes can be added in fields used in WHERE clauses, ORDER BY operations, foreign keys, and JOIN operations. Indexes should be avoided in small tables under 1000 rows, Boolean fields with low cardinality, and tables with heavy write operations.

You can monitor query performance using Django Debug Toolbar in development and PostgreSQL's query logs in production. Add indexes based on actual slow queries, not speculation.

Implementing connection pooling

Establishing database connections is expensive. Each connection requires network handshakes, authentication, and session initialisation. Connection pooling maintains a pool of established connections that can be reused across requests. PostgreSQL's PgBouncer works excellently with Django.

Install PgBouncer as follows:

```
sudo apt-get install pgbouncer
```

Now configure `/etc/pgbouncer/pgbouncer.ini`:

```
[databases]
mydb = host=localhost port=5432 dbname=production_db
[pgbouncer]
listen_port = 6432
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 25
```

Next, update Django settings:

```
DATABASES = {
```